دانشگاه صنعتی شریف

دانشکده مهندسی کامپیوتر

گزارش تمرین و پیاده‌سازی تکلیف ۲

درس مدیریت و مهندسی امنیت اطلاعات

استاد درس: دکتر خرازی

دانشجو: فرهاد توکلیان

شماره دانشجویی: ۴۰۴۲۰۴۸۰۵

بهار ۱۴۰۵

بخش ۲

۲.۱

۲.۱ بررسی مکانیزم‌های حفاظتی موجود در سیستم شخصی

لپ‌تاپ من مبتنی بر UEFI است که از چندین مکانیزم امنیتی برای محافظت از فرایند بوت و داده‌های کاربر استفاده می‌کند. مانند Secure Boot، رمزنگاری کامل دیسک، قابلیت‌های امنیتی سخت‌افزاری و کنترل دسترسی در سطح سیستم‌عامل هستند.

Secure Boot با بررسی امضای دیجیتال اجزای بوت، از اجرای کدهای غیرمجاز در مراحل اولیه راه‌اندازی سیستم جلوگیری می‌کند. نقطه قوت اصلی این فناوری جلوگیری از اجرای Bootkit ها و Rootkit ها است، اما در صورت وجود آسیب‌پذیری در Firmware یا سومدیریت کلیدها ممکن است اثربخشی آن کاهش یابد.

TPM 2.0 یکی دیگر از فناوری‌های مهم امنیتی است که برای ذخیره امن کلیدهای رمزنگاری و اندازه‌گیری زنجیره بوت مورد استفاده قرار می‌گیرد. مهم‌ترین مزیت TPM مقاومت در برابر استخراج کلیدها توسط مهاجم است، اما وابستگی آن به سخت‌افزار و پیچیدگی مدیریت آن از محدودیت‌های این فناوری محسوب می‌شود.

رمزنگاری کامل دیسک نیز نقش مهمی در حفاظت از محرمانگی داده‌ها دارد و در صورت سرقت یا دسترسی فیزیکی غیرمجاز به دستگاه، از افشای اطلاعات جلوگیری می‌کند. با این حال در صورتی که سیستم در حالت روشن یا Unlock شده باشد، این مکانیزم به تنهایی کافی نخواهد بود.

بر اساس روندهای فعلی صنعت، تقریبا تمامی رایانه‌های جدید از UEFI پشتیبانی می‌کنند و بخش بزرگی از آن‌ها به TPM 2.0 و Secure Boot مجهز هستند. همچنین با توجه به الزامات Windows 11، استفاده از این فناوری‌ها نسبت به سال‌های گذشته به شکل قابل توجهی افزایش یافته است. در مقابل، فناوری‌های پیشرفته‌تری مانند Measured Boot و Unified Kernel Image هنوز بیشتر در محیط‌های سازمانی و کاربران حرفه‌ای مشاهده می‌شوند.

۲.۲

Secure Boot یک مکانیزم امنیتی مبتنی بر UEFI است که از اجرای کدهای غیرمجاز در فرآیند بوت جلوگیری می‌کند. در این روش Firmware قبل از اجرای Bootloader، Kernel و سایر اجزای بوت، امضای دیجیتال آن‌ها را بررسی می‌کند. تنها در صورتی که امضا معتبر و متعلق به یک مرجع مورد اعتماد باشد، اجرای آن‌ها مجاز خواهد بود. هدف اصلی Secure Boot جلوگیری از اجرای Bootkit ها، Rootkit ها و سایر بدافزارهایی است که فرآیند بوت را هدف قرار می‌دهند.

در مقابل، Measured Boot رویکرد متفاوتی دارد. این مکانیزم به جای جلوگیری از اجرا، تمامی اجزای زنجیره بوت را اندازه‌گیری کرده و هش آن‌ها را در ثبات‌های PCR موجود در TPM ثبت می‌کند. در نتیجه هرگونه تغییر در Bootloader ، Kernel، Initramfs یا تنظیمات بوت باعث تغییر مقادیر ثبت‌شده خواهد شد. این اطلاعات می‌توانند برای تشخیص دستکاری سیستم یا آزادسازی کلیدهای رمزنگاری مورد استفاده قرار گیرند.

سیستمی که Secure Boot دارد اما فاقد Measured Boot است، تنها می‌تواند از اجرای کدهای فاقد امضای معتبر جلوگیری کند. با این حال در چنین سیستمی امکان ثبت و تشخیص تغییرات رخ داده در زنجیره بوت وجود ندارد. بنابراین برخی حملات پیشرفته که از مؤلفه‌های امضاشده اما آسیب‌پذیر استفاده می‌کنند ممکن است شناسایی نشوند.

به طور کلی Secure Boot و Measured Boot مکمل یکدیگر هستند. Secure Boot وظیفه جلوگیری از اجرای کدهای غیرمجاز را بر عهده دارد، در حالی که Measured Boot امکان اندازه‌گیری، ثبت و اعتبارسنجی وضعیت بوت را فراهم می‌کند. استفاده همزمان از این دو فناوری به همراه TPM و رمزنگاری کامل دیسک، یکی از مؤثرترین راهکارهای دفاع در برابر حملات مرتبط با فرایند بوت محسوب می‌شود.

۲.۳

ابزار systemd-cryptenroll برای ثبت و مدیریت روش‌های احراز هویت در دیسک‌های رمزنگاری‌شده LUKS استفاده می‌شود. این ابزار امکان اتصال کلید رمزنگاری دیسک به TPM، کلیدهای امنیتی سخت‌افزاری یا روش‌های احراز هویت دیگر را فراهم می‌کند. یکی از کاربردهای مهم آن ذخیره کلید رمزنگاری در TPM است تا دیسک تنها در صورتی به‌صورت خودکار باز شود که وضعیت بوت سیستم معتبر باشد.

در برابر حملات Evil Maid، این ابزار با وابسته کردن آزادسازی کلید رمزنگاری به مقادیر PCR ثبت‌شده در TPM عمل می‌کند. اگر مهاجم Bootloader، Kernel یا سایر اجزای بوت را دستکاری کند، مقادیر PCR تغییر کرده و TPM از آزادسازی کلید خودداری خواهد کرد.

نقاط قوت این ابزار شامل یکپارچگی با systemd، افزایش امنیت فرایند بوت و کاهش نیاز به وارد کردن رمز عبور در هر بار راه‌اندازی است. در مقابل، وابستگی به TPM و نیاز به ثبت مجدد کلیدها پس از برخی تغییرات سیستمی از مهم‌ترین محدودیت‌های آن محسوب می‌شوند.

ابزار systemd-pcrlock برای تعریف و مدیریت وضعیت‌های مجاز PCR در TPM طراحی شده است. این ابزار مشخص می‌کند که چه مقادیر یا ترکیب‌هایی از PCR ها قابل اعتماد هستند و در چه شرایطی دسترسی به کلیدهای رمزنگاری مجاز خواهد بود.

در سناریوی حملهEvil Maid ، هرگونه تغییر در زنجیره بوت باعث تغییر مقادیر PCR می‌شود. در نتیجه اگر وضعیت جدید با مقادیر مجاز تعریف‌شده توسط systemd-pcrlock مطابقت نداشته باشد، کلیدهای محافظت‌شده آزاد نخواهند شد و مهاجم قادر به دسترسی به داده‌های رمزنگاری‌شده نخواهد بود.

از مهم‌ترین مزایای systemd-pcrlock می‌توان به کنترل دقیق وضعیت‌های بوت مجاز و افزایش مقاومت در برابر دستکاری فیزیکی اشاره کرد. با این حال پیکربندی آن نسبتاً پیچیده است و برای عملکرد صحیح به TPM و Measured Boot وابستگی دارد.

۲.۴

امروزه حملات Evil Maid همچنان یکی از تهدیدهای جدی برای امنیت سیستم‌ها محسوب می‌شوند، زیرا در این سناریو مهاجم به دستگاه دسترسی فیزیکی پیدا کرده و می‌تواند اجزای فرایند بوت را دستکاری کند. در صورت نبود مکانیزم‌های حفاظتی مناسب، مهاجم قادر خواهد بود Bootloader یا Kernel را تغییر داده و اطلاعات حساسی مانند گذرواژه‌ها یا کلیدهای رمزنگاری را سرقت کند.

مؤثرترین راهکارهای حفاظتی در سیستم‌های مدرن ترکیبی ازSecure Boot ، Measured Boot، TPM 2.0 و رمزنگاری کامل دیسک هستند Secure Boot .از اجرای کدهای غیرمجاز جلوگیری می‌کند، Measured Boot تغییرات زنجیره بوت را ثبت می‌نماید و TPM امکان محافظت از کلیدهای رمزنگاری را فراهم می‌سازد. ابزارهایی مانند systemd-cryptenroll و systemd-pcrlock نیز با وابسته کردن دسترسی به داده‌ها به وضعیت معتبر سیستم، سطح امنیت را افزایش می‌دهند.

با وجود این پیشرفت‌ها، همچنان چالش‌هایی باقی مانده است. بسیاری از کاربران این قابلیت‌ها را فعال نمی‌کنند، برخی سیستم‌ها فاقد TPM هستند و آسیب‌پذیری‌های Firmware نیز می‌توانند زنجیره اعتماد را تضعیف کنند. بنابراین امنیت در برابر حملات Evil Maid نیازمند استفاده همزمان از چندین لایه حفاظتی است و هیچ راهکار واحدی به تنهایی کافی نخواهد بود.

بخش ۳

۳.۱ نصب Arch Linux

فایل ISO آرچ لینوکس دریافت و روی ماشین مجازی UTM اجرا شد. از آنجایی که فایل ISO نسخه x86_64 بود، در حالت شبیه‌ساز اجرا شد. سیستم در حالت UEFI بوت و نصب شد. در طول نصب، پارتیشن‌بندی دیسک، نصب بسته‌های پایه سیستم و پیکربندی اولیه سیستم‌عامل صورت گرفت.



تنظیم پسورد کاربر روت



پارتیشن‌بندی دیسک



آغاز نصب بعد از انجام پیکربندی

```
              liblsan-16.1.1+r12+g301eb08fa2c5-1  libmakepkg-dropins-20-1
              libmnl-1.0.5-2  libnetfilter_conntrack-1.1.1-1
              libnfnetlink-1.0.2-2  libnftnl-1.3.1-1  libnghttp2-1.69.0-1
              libnghttp3-1.16.0-1  libngtcp2-1.23.0-1  libnl-3.12.0-1
              libnsl-2.0.1-2  libobjc-16.1.1+r12+g301eb08fa2c5-1
              libp11-kit-0.26.2-1  libpcap-1.10.6-1  libpsl-0.21.5-2
              libquadmath-16.1.1+r12+g301eb08fa2c5-1  libsasl-2.1.28-5
              libseccomp-2.6.0-1  libsecret-0.21.7-1  libssh2-1.11.1-1
              libstdc++-16.1.1+r12+g301eb08fa2c5-1  libsysprof-capture-50.0-2
              libtasn1-4.21.0-1  libtirpc-1.3.7-1
              libtsan-16.1.1+r12+g301eb08fa2c5-1
              libubsan-16.1.1+r12+g301eb08fa2c5-1  libunistring-1.4.2-1
              libusb-1.0.30-1  libverto-0.3.2-6  libxcrypt-4.5.2-1
              libxml2-2.15.3-1  licenses-20240728-1  linux-api-headers-6.19-1
              linux-firmware-amdgpu-20260519-1
              linux-firmware-atheros-20260519-1
              linux-firmware-broadcom-20260519-1
              linux-firmware-cirrus-20260519-1
              linux-firmware-intel-20260519-1
              linux-firmware-mediatek-20260519-1
              linux-firmware-nvidia-20260519-1
              linux-firmware-other-20260519-1
              linux-firmware-radeon-20260519-1
              linux-firmware-realtek-20260519-1
              linux-firmware-whence-20260519-1  lmdb-0.9.35-1  lz4-1:1.10.0-2
              mkinitcpio-41-4  mkinitcpio-busybox-1.36.1-1  mpfr-4.2.2-1
              ncurses-6.6-2  nettle-4.0-1  nftables-1:1.1.6-3  npth-1.8-1
              openssl-3.6.2-2  p11-kit-0.26.2-1  pacman-7.1.0.r9.g54d9411-2
              pacman-mirrorlist-20260406-1  pam-1.7.2-2  pambase-20250719-1
              pciutils-3.15.0-1  pcre2-10.47-1  pinentry-1.3.2-2  popt-1.19-2
              procps-ng-4.0.6-1  psmisc-23.7-2  readline-8.3.003-1  sed-4.10-1
              shadow-4.18.0-1  sqlite-3.53.1-1  systemd-260.2-2
              systemd-libs-260.2-2  systemd-sysvcompat-260.2-2  tar-1.35-2
              tpm2-tss-4.1.3-1  tzdata-2026b-1  util-linux-2.42.1-1
              util-linux-libs-2.42.1-1  xz-5.8.3-1  zlib-1:1.3.2-3
              zstd-1.5.7-3  base-3-3  linux-7.0.10.arch1-1
              linux-firmware-20260519-1  sudo-1.9.17.p2-2

Total Download Size:    649.36 MiB
Total Installed Size:  1067.37 MiB

:: Proceed with installation? [Y/n]
:: Retrieving packages...
 linux-7.0.10.arc...    60.2 MiB   626 KiB/s 02:22 [########----------]  40%
 linux-firmware-i...     8.3 MiB   107 KiB/s 18:30 [#-------------------]   6%
 linux-firmware-n...     8.3 MiB  93.1 KiB/s 16:35 [#-------------------]   8%
 linux-firmware-a...     8.2 MiB  93.3 KiB/s 07:49 [###-----------------]  16%
 linux-firmware-o...     8.2 MiB   101 KiB/s 03:34 [######--------------]  28%
 Total (  0/153)        93.2 MiB   961 KiB/s 09:52 [###-----------------]  14%
```

دانلود و نصب پکیج‌ها

```
[root@archlinux FlashSystems-ukibak-891e789]# cat /etc/os-release
NAME="Arch Linux"
PRETTY_NAME="Arch Linux"
ID=arch
BUILD_ID=rolling
ANSI_COLOR="38;2;23;147;209"
HOME_URL="https://archlinux.org/"
DOCUMENTATION_URL="https://wiki.archlinux.org/"
SUPPORT_URL="https://bbs.archlinux.org/"
BUG_REPORT_URL="https://gitlab.archlinux.org/groups/archlinux/-/issues"
PRIVACY_POLICY_URL="https://terms.archlinux.org/docs/privacy-policy/"
LOGO=archlinux-logo
[root@archlinux FlashSystems-ukibak-891e789]# _
```

اجرای دستور cat /etc/os-release

```
[root@archlinux FlashSystems-ukibak-891e789]# uname -a
Linux archlinux 7.0.11-arch1-1 #1 SMP PREEMPT_DYNAMIC Tue, 02 Jun 2026 18:26:58 +0000 x86_64 GNU/Linux
[root@archlinux FlashSystems-ukibak-891e789]# _
```

خروجی دستور uname -a

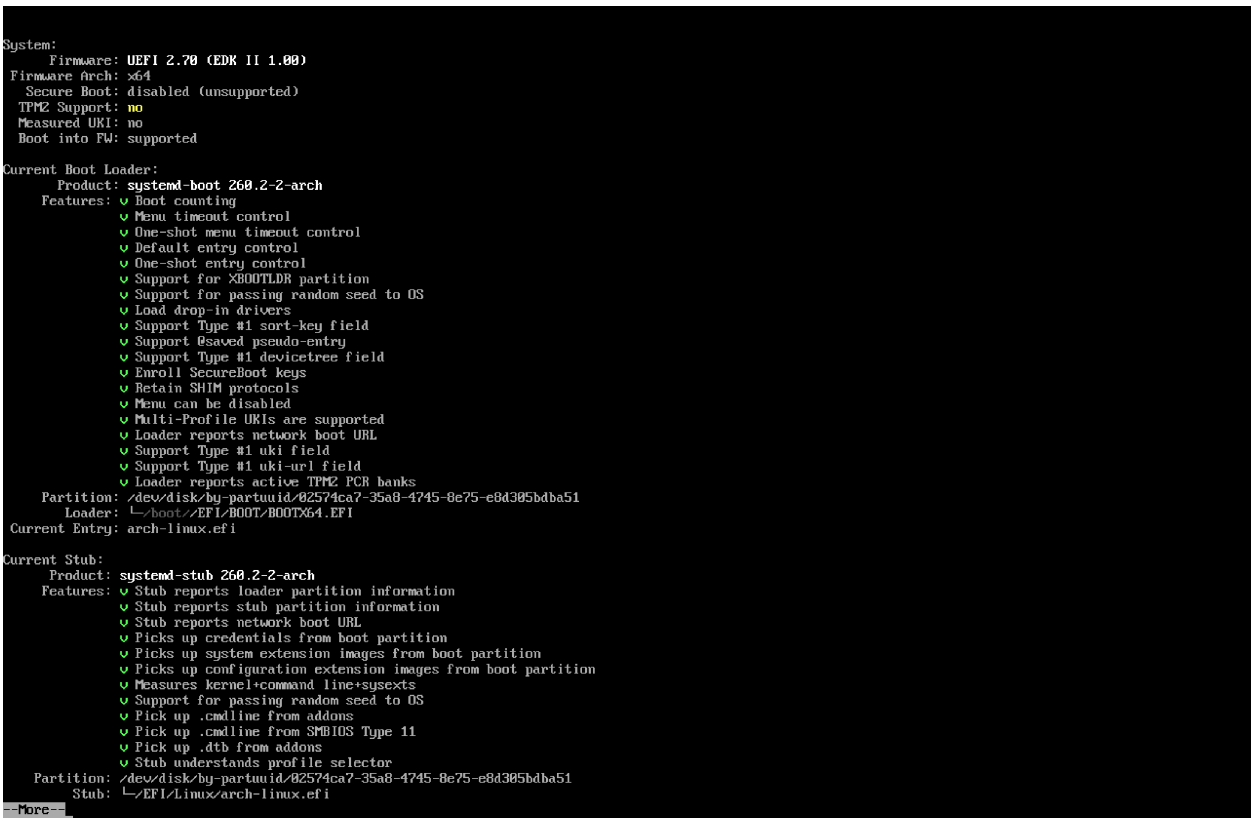


خروجی دستور lsblk

۳.۲ راه‌اندازی Unified Kernel Image (UKI)

در این مرحله سیستم Arch Linux برای استفاده از Unified Kernel Image (UKI) پیکربندی و بررسی شد. هدف UKI ترکیب کرنل لینوکس، فایل initramfs و پارامترهای خط فرمان کرنل در یک فایل EFI واحد است. این رویکرد باعث ساده‌تر شدن زنجیره بوت و افزایش یکپارچگی فرایند راه‌اندازی سیستم می‌شود.

برای بررسی وضعیت سیستم از دستور bootctl status استفاده شد:



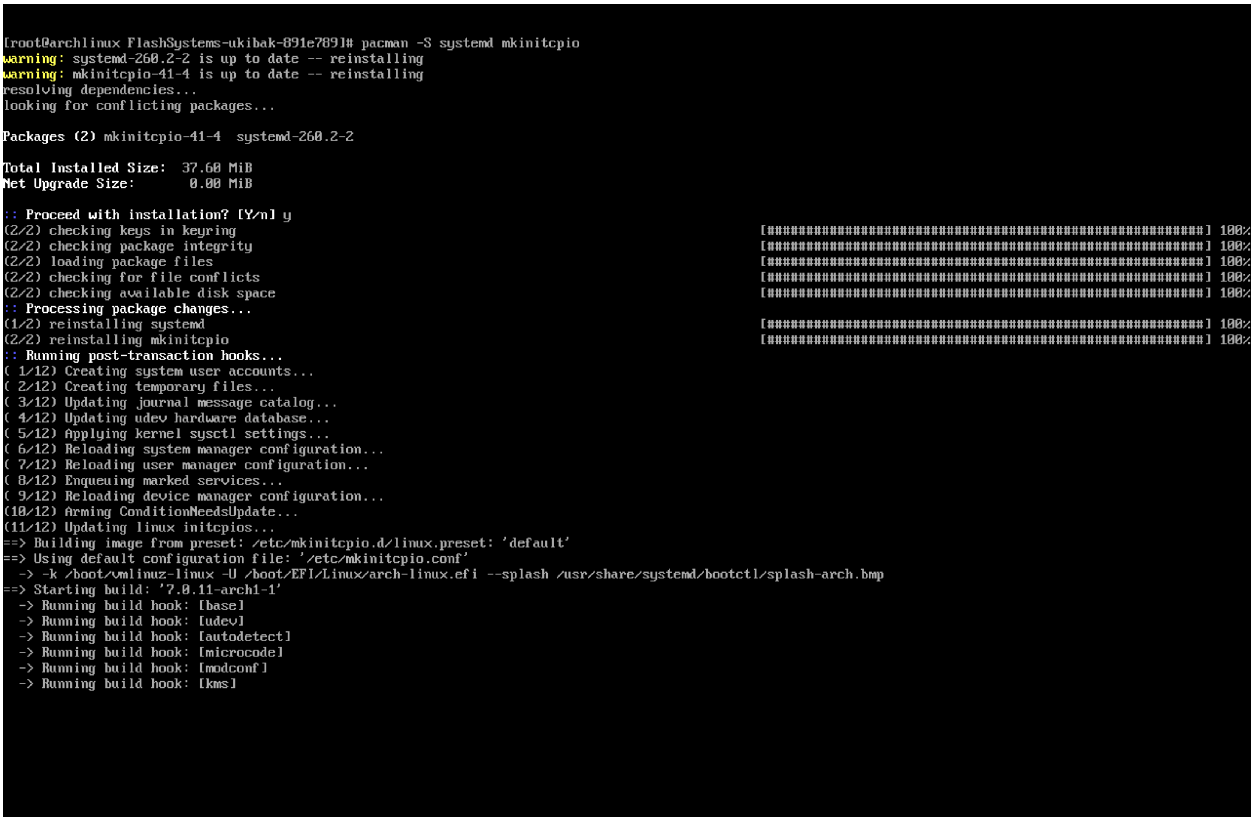
اجرای دستور bootctl status

خروجی نشان داد که سیستم در حالت UEFI اجرا شده و از systemd-boot به عنوان Boot Loader استفاده می‌کند. همچنین ورودی فعال بوت به فایل arch-linux.efi اشاره می‌کند که یک Unified Kernel Image محسوب می‌شود. وجود فایل arch-linux.efi در مسیر /EFI/Linux/ و استفاده مستقیم Boot Loader از این فایل نشان می‌دهد که سیستم با موفقیت برای بوت از طریق UKI پیکربندی شده است.

ماشین مجازی در UTM روی Apple Silicon اجرا شد. Firmware مجازی مورد استفاده از Secure Boot و TPM2 پشتیبانی نمی‌کرد، و امکان فعال‌سازی این قابلیت‌ها در محیط آزمایش وجود نداشت.

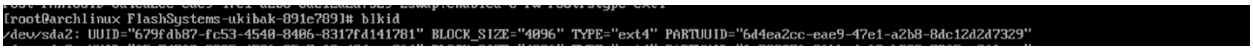
فرض می‌شود Arch خام نصب شده و UKI هنوز تنظیم نشده و مراحل نصب را مجددا انجام می‌شود.

۳.۲.۱ نصب mkinitcpio، برای ساخت initramfs و در تولید Unified Kernel Image.



۳.۲.۲ ایجاد Kernel Command Line

پیدا کردن PARTUUID: با استفاده از دستور blkid شناسه پارتیشن ریشه (PARTUUID) استخراج و در فایل /etc/kernel/cmdline قرار داده شد تا کرنل در زمان بوت بتواند پارتیشن ریشه را به‌درستی شناسایی و mount کند.



ایجاد فایل Kernel Command Line



۳.۲.۳. پیکربندی mkinitcpio برای تولید UKI

باز کردن فایل linux.preset



اجرای دستور nano /etc/mkinitcpio.d/linux.preset

در این فایل مقدار

default_image="/boot/initramfs-linux.img "

را به

PRESETS=('default')

default_uki="/boot/EFI/Linux/arch-linux.efi"

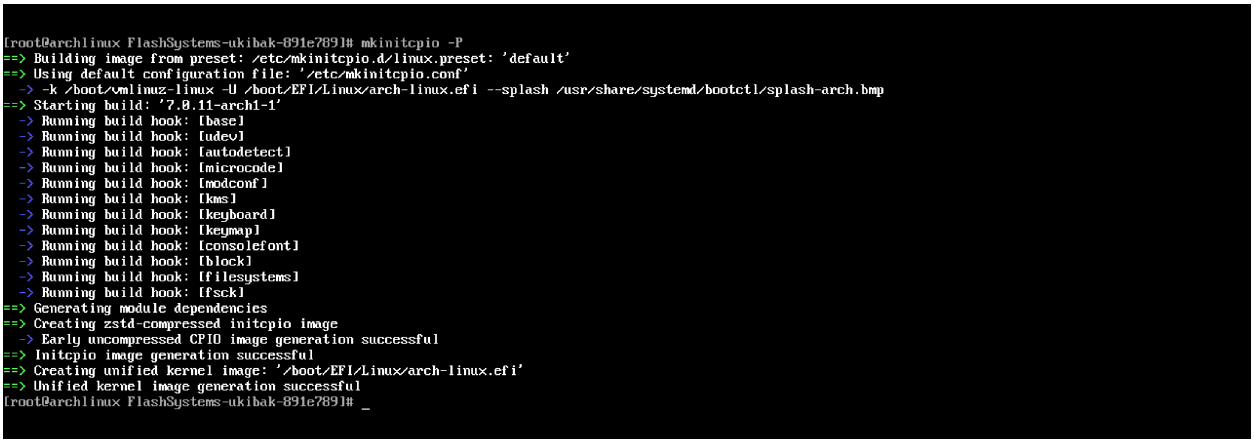
تغییر می‌دهیم.

۳.۲.۴ ساخت پوشه EFI Linux



اجرای دستور mkdir -p /boot/EFI/Linux

۳.۲.۵ تولید UKI



اجرای دستور mkinitcpio -P



اجرای دستور ls -lh /boot/EFI/Linux

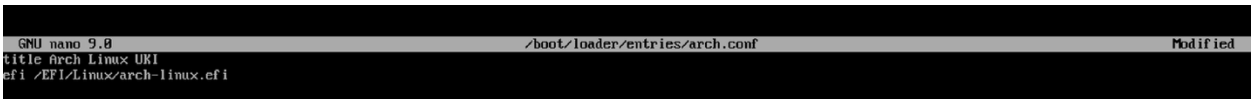
۳.۲.۶ نصب systemd-boot



اجرای دستور bootctl install

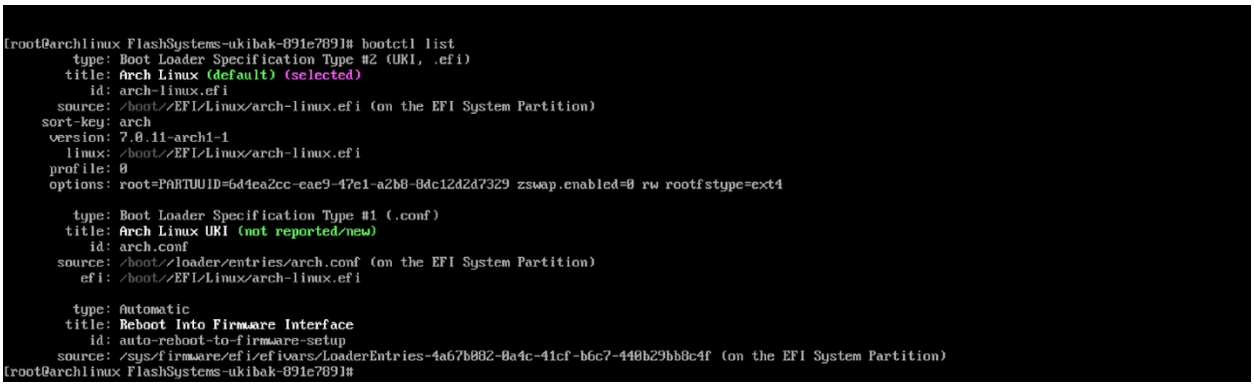
۳.۲.۷ ساخت Boot Entry

ساخت فایل arch.conf



اجرای دستور nano /boot/loader/entries/arch.conf نوشتن محتوای فایل

۳.۲.۸ بررسی



در خروجی دو نوع Entry وجود دارد:

۱. UKI خودکار همان چیزی که Arch ساخته.

type: Boot Loader Specification Type #2 (UKI, .efi)

title: Arch Linux (default) (selected)

id: arch-linux.efi

source: /boot/EFI/Linux/arch-linux.efi

۲. Entry که ما دستی ساختیم.

type: Boot Loader Specification Type #1 (.conf)

title: Arch Linux UKI

id: arch.conf

که هر دو در آخر به /boot/EFI/Linux/arch-linux.efi/ می‌رسند.

بخش ۳.۳

نصب rustc و cargo

```
-bash: cargo: command not found
[root@archlinux FlashSystems-ukibak-891e789]# rust --version
-bash: rust: command not found
[root@archlinux FlashSystems-ukibak-891e789]# pacman -S rust
resolving dependencies...
looking for conflicting packages...

Packages (10) compiler-rt-22.1.6-1  gcc-16.1.1+r12+g301eb08fa2c5-1  libedit-20260512_3.1-1  libgit2-1:1.9.4-1  libisl-0.27-1  libmpc-1.4.1-1  lld-22.1.6-1
              llhttp-9.3.1-1  llvm-libs-22.1.6-1  rust-1:1.96.0-1

Total Download Size:   101.34 MiB
Total Installed Size:  727.95 MiB

:: Proceed with installation? [Y/n] y
:: Retrieving packages...
 lld-22.1.6-1-x86_64                                    2.1 MiB   246 KiB/s 00:09 [#####################################################] 100%
 libgit2-1:1.9.4-1-x86_64                            1016.3 KiB   308 KiB/s 00:03 [#####################################################] 100%
 libisl-0.27-1-x86_64                                 886.4 KiB   345 KiB/s 00:03 [#####################################################] 100%
 compiler-rt-22.1.6-1-x86_64                            4.6 MiB   308 KiB/s 00:15 [#####################################################] 100%
 libedit-20260512_3.1-1-x86_64                        117.5 KiB   136 KiB/s 00:01 [#####################################################] 100%
 libmpc-1.4.1-1-x86_64                                 91.9 KiB  55.5 KiB/s 00:02 [#####################################################] 100%
 llhttp-9.3.1-1-x86_64                                 30.8 KiB  52.9 KiB/s 00:01 [#####################################################] 100%
 llvm-libs-22.1.6-1-x86_64                             41.0 MiB   373 KiB/s 01:52 [#####################################################] 100%
 gcc-16.1.1+r12+g301eb08fa2c5-1-x86_64                 57.3 MiB   416 KiB/s 02:21 [#####################################################] 100%
 rust-1:1.96.0-1-x86_64                                74.3 MiB   462 KiB/s 02:44 [#####################################################] 100%
 Total (10/10)                                        101.3 MiB  1128 KiB/s 02:45 [#####################################################] 100%
(10/10) checking keys in keyring                                               [#####################################################] 100%
(10/10) checking package integrity                                             [#####################################################] 100%
(10/10) loading package files                                                  [#####################################################] 100%
(10/10) checking for file conflicts                                            [#####################################################] 100%
(10/10) checking available disk space                                          [#####################################################] 100%
:: Processing package changes...
( 1/10) installing compiler-rt                                                 [#####################################################] 100%
( 2/10) installing libisl                                                      [#####################################################] 100%
( 3/10) installing libmpc                                                      [#####################################################] 100%
( 4/10) installing gcc                                                         [#####################################################] 100%
Optional dependencies for gcc
    lib32-gcc-libs: for generating code for 32-bit ABI
( 5/10) installing llhttp                                                      [#####################################################] 100%
( 6/10) installing libgit2                                                     [#####################################################] 100%
( 7/10) installing libedit                                                     [#####################################################] 100%
( 8/10) installing llvm-libs                                                   [#####################################################] 100%
( 9/10) installing lld                                                         [#####################################################] 100%
(10/10) installing rust                                                        [#####################################################] 100%
Optional dependencies for rust
    gdb: rust-gdb script
    lldb: rust-lldb script
:: Running post-transaction hooks...
(1/1) Arming ConditionNeedsUpdate...
[root@archlinux FlashSystems-ukibak-891e789]# _
```

اجرای دستور pacman -S rust

۳.۳.۱ Build کردن برنامه

```
[root@archlinux FlashSystems-ukibak-891e789]# cargo build --release
    Updating crates.io index
  Downloaded clap_lex v1.1.0
  Downloaded const-fnv1a-hash v1.1.0
  Downloaded heck v0.5.0
  Downloaded anstyle v1.0.14
  Downloaded simple_logger v5.2.0
  Downloaded quote v1.0.45
  Downloaded thiserror-impl v2.0.18
  Downloaded thiserror v2.0.18
  Downloaded unicode-ident v1.0.24
  Downloaded proc-macro2 v1.0.106
  Downloaded log v0.4.29
  Downloaded clap v4.6.1
  Downloaded clap_derive v4.6.1
  Downloaded byteorder v1.5.0
  Downloaded clap_builder v4.6.0
  Downloaded syn v2.0.117
  Downloaded libc v0.2.186
  Downloaded 17 crates (1.6MiB) in 2.74s
   Compiling proc-macro2 v1.0.106
   Compiling quote v1.0.45
   Compiling unicode-ident v1.0.24
   Compiling syn v2.0.117
   Compiling clap_lex v1.1.0
   Compiling heck v0.5.0
   Compiling libc v0.2.186
   Compiling thiserror v2.0.18
   Compiling anstyle v1.0.14
   Compiling clap_builder v4.6.0
   Compiling clap_derive v4.6.1
   Compiling thiserror-impl v2.0.18
   Compiling log v0.4.29
   Compiling simple_logger v5.2.0
   Compiling clap v4.6.1
   Compiling const-fnv1a-hash v1.1.0
   Compiling byteorder v1.5.0
   Compiling ukibak v1.1.5 (/root/FlashSystems-ukibak-891e789)
    Finished `release` profile [optimized] target(s) in 3m 01s
[root@archlinux FlashSystems-ukibak-891e789]# _
```

اجرای دستور cargo build --release

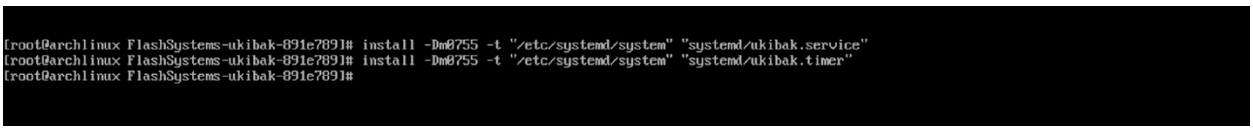
۳.۳.۲ نصب باینری



اجرای دستور install -Dm0750 -t "/usr/sbin/" "target/release/ukibak"

این دستور برنامه را در مسیر استاندارد سیستم نصب می‌کند.

۳.۳.۳ نصب سرویس و تایمر



اجرای دستورات

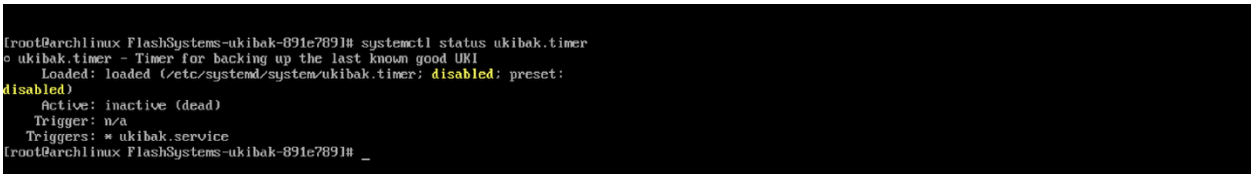
install -Dm0755 -t "/etc/systemd/system" "systemd/ukibak.service"

install -Dm0755 -t "/etc/systemd/system" "systemd/ukibak.timer"

ukibak.service یک بار از UKI بکاپ می‌گیرد.

ukibak.timer ۵ دقیقه بعد از بوت ukibak.service را اجرا می‌کند.

۳.۳.۴ بررسی وضعیت تایمر



اجرای دستور systemctl status ukibak.timer

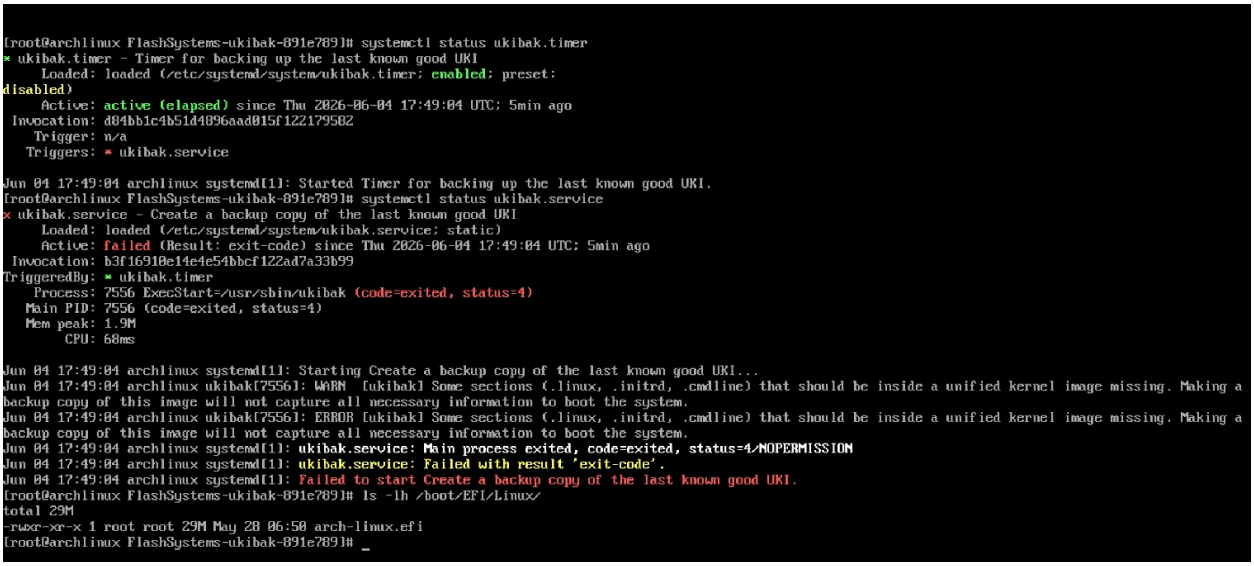
فایل تایمر درست نصب شده ولی هنوز فعال نشده. از آنجایی که هست disabled در بوت‌های بعدی هم اجرا نخواهد شد.

فعال کردن تایمر



اجرای دستور systemctl enable --now ukibak.timer

بررسی مجدد وضعیت تایمر



فایل ukibak

/boot/EFI/Linux/arch-linux.efi

را بررسی کرده و نتیجه گرفته که این فایل از دید خودش یک UKI کامل نیست.

تحلیل خطای ukibak

هنگام فعال‌سازی سرویس ukibak انتظار داشتیم ابزار از UKI فعلی سیستم نسخه پشتیبان تهیه کند:
/boot/EFI/Linux/arch-linux.efi

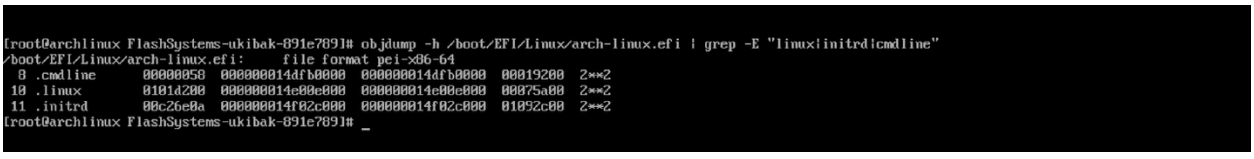
اما اجرای سرویس با خطا متوقف شد:

Some sections (.linux, .initrd, .cmdline) that should be inside a unified kernel image missing

در نگاه اول این خطا نشان می‌دهد که فایل بوت‌شده یک UKI معتبر نیست.

بررسی ساختار UKI

برای اطمینان از معتبر بودن UKI، فایل EFI بررسی شد:

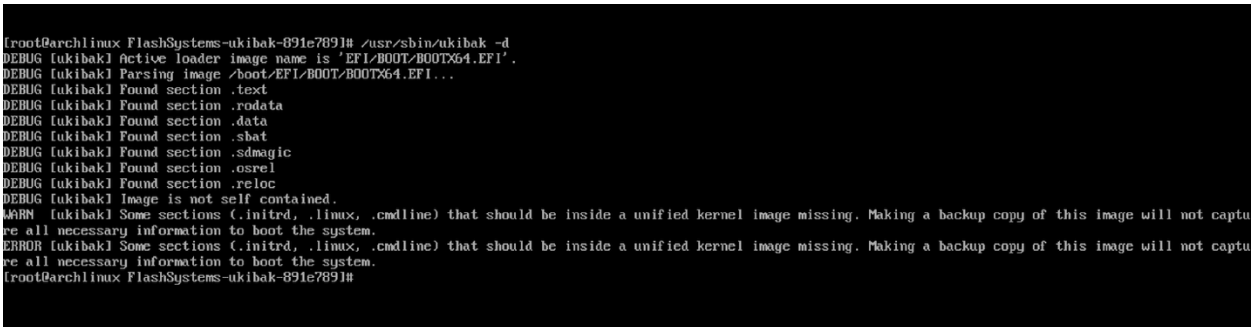


اجرای دستور objdump -h /boot/EFI/Linux/arch-linux.efi

خروجی وجود بخش‌های cmdline، linux و initrd را تایید کرد. بنابراین فایل arch-linux.efi یک UKI معتبر است.

اجرای ابزار در حالت Debug

برای یافتن علت خطا، برنامه با گزینه Debug اجرا شد:



اجرای دستور /usr/sbin/ukibak -d

علت خطا

ابزار ukibak به جای بررسی فایل UKI واقعی:

/boot/EFI/Linux/arch-linux.efi

فایل زیر را بررسی می‌کرد:

/boot/EFI/BOOT/BOOTX64.EFI

این فایل در واقع:

systemd-boot bootloader

است و UKI نیست.

در نتیجه هنگام بررسی آن، بخش‌های ، linux و initrd پیدا نشدند و برنامه تصور کرد فایل بوت‌شده یک UKI ناقص است.

بررسی وضعیت بوت



اجرای دستور bootctl status

خروجی این دستور نشان داد که فرایند بوت به شکل

1. UEFI Firmware
2. BOOTX64.EFI (systemd-boot)
3. arch-linux.efi (UKI)

انجام می‌شود. ابزار ukibak به اشتباه فایل مرحله اول (BOOTX64.EFI) را به عنوان تصویر بوت‌شده تشخیص می‌دهد، در حالی که UKI واقعی در مرحله دوم قرار دارد.

نتیجه‌گیری از خطا:

این خطا ناشی از خراب بودن UKI نبود.

بررسی‌های انجام‌شده نشان داد:

- UKI به‌درستی ساخته شده است.
- فایل EFI شامل کرنل، initramfsو خط فرمان کرنل است.
- سیستم با موفقیت از UKI بوت می‌شود.
- مشکل ناشی از نحوه تشخیص فایل بوت‌شده توسط ukibak در محیط آزمایش است.

در نتیجه فرآیند پشتیبان‌گیری خودکار متوقف شد، اما ساختار UKI و پیکربندی بوت سیستم صحیح بودند.

بکاپ‌گیری در ادامه دستی انجام می‌شود.

۳.۳.۵ بکاپ‌گیری

```
[root@archlinux FlashSystems-ukibak-891e7891]# cp /boot/EFI/Linux/arch-linux.efi  \
> /boot/EFI/Linux/linux-last.efi
[root@archlinux FlashSystems-ukibak-891e7891]#
```

اجرای دستور cp /boot/EFI/Linux/arch-linux.efi /boot/EFI/Linux/linux-last.efi

مشاهده نسخه اصلی و نسخه پشتبان

```
[root@archlinux FlashSystems-ukibak-891e7891]# ls -lh /boot/EFI/Linux/
total 58M
-rwxr-xr-x 1 root root 29M May 28 06:50 arch-linux.efi
-rwxr-xr-x 1 root root 29M Jun  4 18:45 linux-last.efi
[root@archlinux FlashSystems-ukibak-891e7891]#
```

اجرای دستور ls -lh /boot/EFI/Linux

۳.۳.۶ ساخت Boot Entry برای نسخه پشتیبان

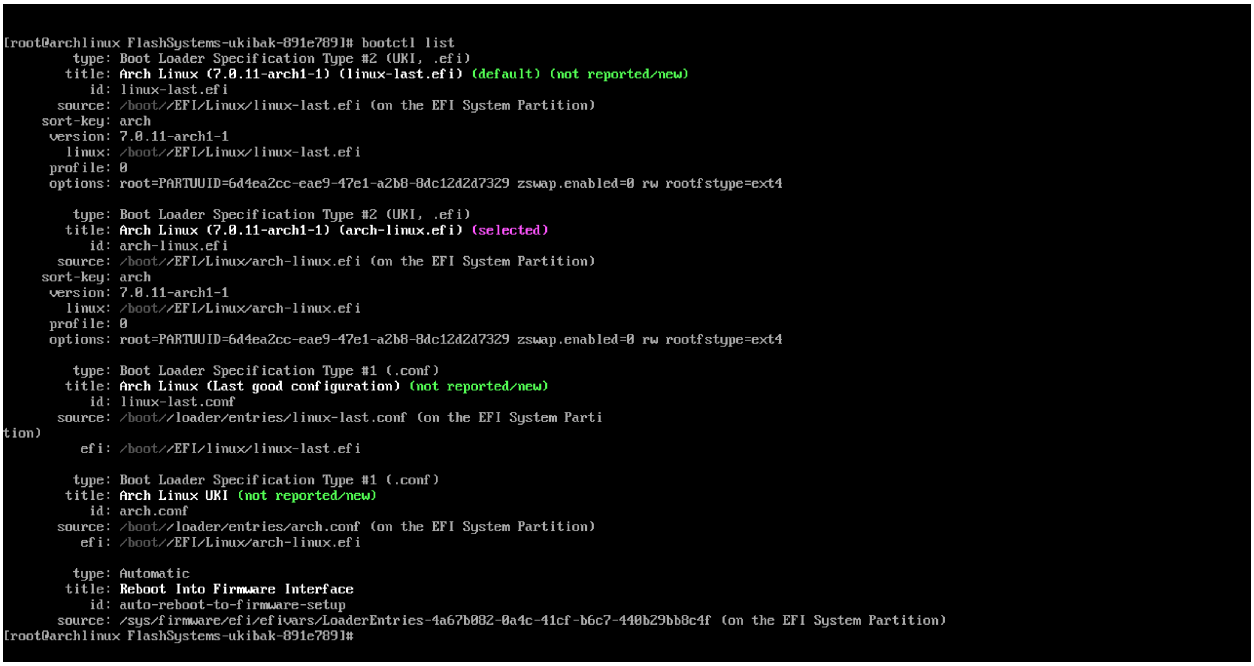
ساخت فایل

```
  GNU nano 9.0                    /boot/loader/entries/linux-last.conf                    Modified
title Arch Linux (Last good configuration)
efi /EFI/linux/linux-last.efi
```

اجرای دستور nano /boot/loader/entries/linux-last.conf و نوشتن محتوای آن

بخش ۳.۴

با اجرای دستور bootctl list مشاهده می‌شود علاوه بر بکاپ دستی که ساخته شد یک بکاپ خودکار نیز ساخته شده‌است!!! (علی‌رغم مشکل موجود در بخش ۳.۳)



اجرای دستور bootctl list

خراب کردن فایل arch-linux.efi



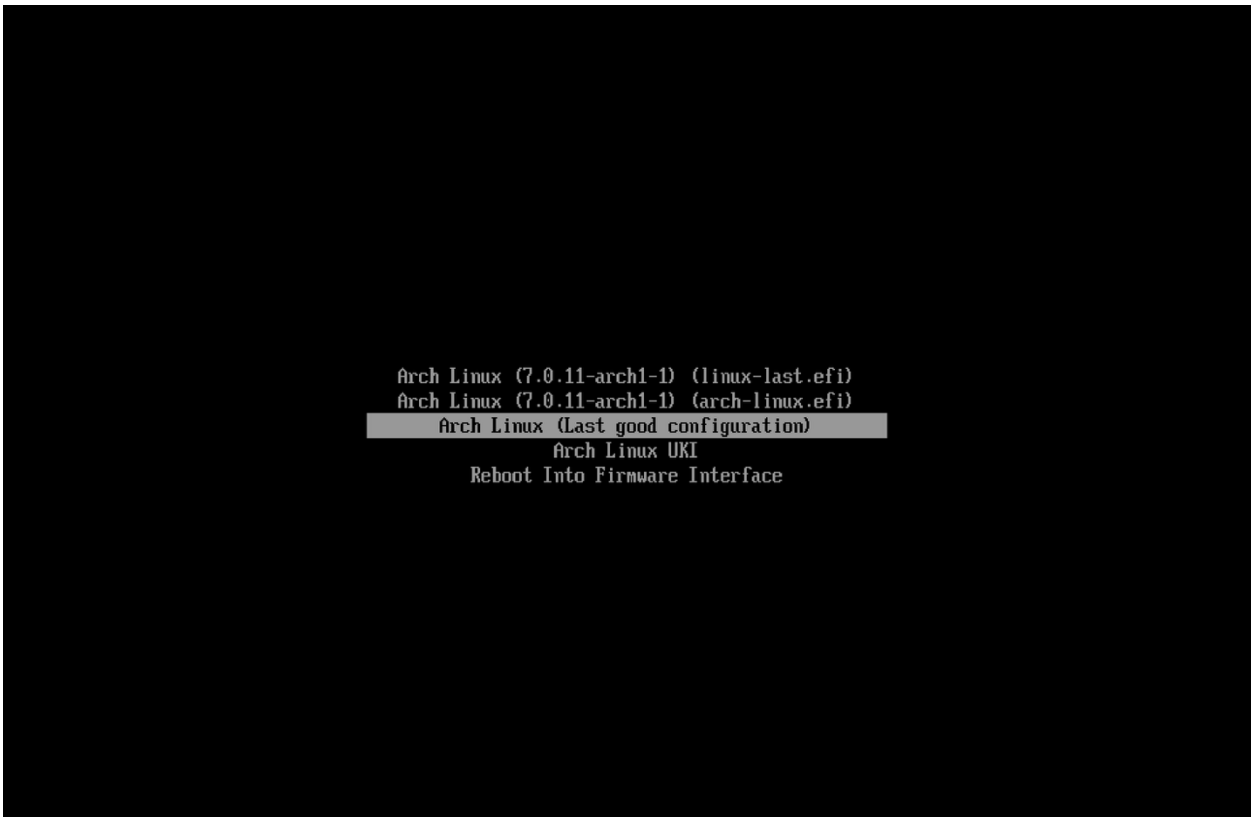
اجرای دستور truncate -s 1M /boot/EFI/Linux/arch-linux.efi

ریبوت کردن



اجرای کامند reboot

در منوی systemd-boot گزینه Arch Linux (Last good configuration) را انتخاب کرده.



لود کردن با Arch Linux (Last good configuration)

سیستم با موفقیت لود شد.



ترمینال بعد از لود

بازگردانی فایل اصلی



اجرا دستور cp /boot/EFI/Linux/linux-last.efi /boot/EFI/Linux/arch-linux.efi

بخش ۳.۵

ابزار ukibak با ایجاد یک نسخه پشتیبان قابل بوت از Unified Kernel Image (UKI)، روشی ساده و کاربردی برای بازیابی سیستم پس از خرابی یا دستکاری فرایند بوت ارائه می‌دهد. از آنجا که UKI شامل کرنل، initramfs و خط فرمان کرنل در یک فایل EFI واحد است، تهیه نسخه پشتیبان از آن به معنای ذخیره کامل پیکربندی بوت سیستم خواهد بود. در صورت بروز مشکل، کاربر می‌تواند مستقیماً از نسخه پشتیبان بوت شده و سیستم را بازیابی کند.

یکی از مهم‌ترین مزایای ukibak سادگی فرآیند بازیابی است. همچنین استفاده از تایمر systemd امکان تهیه خودکار نسخه پشتیبان از آخرین پیکربندی سالم سیستم را فراهم می‌کند. این موضوع به‌ویژه پس از به‌روزرسانی کرنل یا تغییر تنظیمات بوت اهمیت زیادی دارد.

با این حال، این رویکرد محدودیت‌هایی نیز دارد. مهم‌ترین محدودیت آن وابستگی کامل به UKI است و در سیستم‌هایی که از ساختار سنتی Bootloader و initramfs استفاده می‌کنند قابل استفاده نیست. همچنین ukibak به تنهایی از وقوع حمله Evil Maid جلوگیری نمی‌کند، بلکه تنها امکان بازیابی سیستم پس از شناسایی یا وقوع حمله را فراهم می‌سازد.

به طور کلی، ukibak مقاومت سیستم را در برابر پیامدهای حملات Evil Maid افزایش می‌دهد و فرآیند بازیابی را ساده‌تر می‌کند، اما برای دستیابی به امنیت کامل باید در کنار فناوری‌هایی مانند Secure Boot، Measured Boot و TPM مورد استفاده قرار گیرد. در غیر این صورت همچنان امکان دستکاری اجزای بوت توسط مهاجمی که دسترسی فیزیکی به سیستم دارد وجود خواهد داشت.